

Week 3 module

```
// 1. Divide the array into two halves.
// 2. Recursively sort the left half.
// 3. Recursively sort the right half.
// 4. Merge both sorted halves.
// 5. Repeat until the entire array is sorted.
// Time Complexity:  $O(n \log n)$ 
// Space Complexity:  $O(n)$ 
```

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}
```

```

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    cin >> n;

    int arr[n];

    for (int i = 0; i < n; i++)
        cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

```

PS C:\Users\Induleka\Desktop\DSA\collage_training> g++ search_algo_2.cpp -o search_algo_2
lage_training
Enter the number of elements: 6
Enter element 1: 450
Enter element 2: 1200
Enter element 3: 800
Enter element 4: 650
Enter element 5: 300
Enter element 6: 1500
300 450 650 800 1200 1500
PS C:\Users\Induleka\Desktop\DSA\collage_training> 

```

```

//Use Merge Sort to sort the array. After sorting, the median is the middle element
//(or average of two middle elements for even N).
// Count elements greater than the median and subtract the smallest element from the largest.

//Algorithm:

//Sort the array using Merge Sort.
//Print the sorted array.
//Find the median.
//Count elements greater than the median.
//Calculate Maximum - Minimum.

//Time Complexity: O(n log n)
//Space Complexity: O(n)

```

```

#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++){
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++){
        R[j] = arr[mid + 1 + j];
    }

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]){
            arr[k++] = L[i++];
        }
        else{
            arr[k++] = R[j++];
        }
    }

    while (i < n1){
        arr[k++] = L[i++];
    }
    while (j < n2){
        arr[k++] = R[j++];
    }
}

```

```

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    cout<<"enter n: ";
    int n;
    cin >> n;

    int arr[n];

    cout<<"enter elements: ";
    for (int i = 0; i < n; i++){
        cin >> arr[i];
    }

    mergeSort(arr, 0, n - 1);

    // Print sorted array
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }
    cout << endl;

    // Find median
    double median;
    if (n % 2 == 1)
        median = arr[n / 2];
    else
        median = (arr[n / 2 - 1] + arr[n / 2]) / 2.0;

    cout << "Median: " << median << endl;
}

```

```

91     // Count orders above median
92     int count = 0;
93     for (int i = 0; i < n; i++) {
94         if (arr[i] > median)
95             count++;
96     }
97
98     cout << "Orders Above Median: " << count << endl;
99
100    // Difference between max and min
101    cout << "Difference: " << arr[n - 1] - arr[0] << endl;
102
103    return 0;
104 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Induleka\Desktop\DSA\collage_training> .\search_algo_2.exe
Enter the number of elements: 7
Enter element 1: 12
Enter element 2: 15
Enter element 3: 18
Enter element 4: 22
Enter element 5: 25
Enter element 6: 30
Enter element 7: 40
12 15 18 22 25 30 40

```

1. Read the number of elements **N**.
2. Read all array elements.
3. Choose the last element as the pivot.
4. Partition the array:
 - Place all elements smaller than the pivot on its left.
 - Place all larger elements on its right.
5. Recursively apply Quick Sort on:
 - Left subarray
 - Right subarray
6. Print the sorted array.

```
search_algo_2.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  // Function to partition the array
5  int partition(int arr[], int low, int high)
6  {
7      int pivot = arr[high];
8      int i = low - 1;
9
10     for(int j = low; j < high; j++)
11     {
12         if(arr[j] < pivot)
13         {
14             i++;
15             swap(arr[i], arr[j]);
16         }
17     }
18
19     swap(arr[i + 1], arr[high]);
20
21     return i + 1;
22 }
23
24 // Quick Sort function
25 void quickSort(int arr[], int low, int high)
26 {
27     if(low < high)
28     {
29         int pi = partition(arr, low, high);
30
31         quickSort(arr, low, pi - 1);
32         quickSort(arr, pi + 1, high);
33     }
34 }
35
36 int main()
37 {
38     int n;
39     cin >> n;
40
41     int arr[n];
42
43     for(int i = 0; i < n; i++)
44         cin >> arr[i];
45
46     quickSort(arr, 0, n - 1);
47
48     for(int i = 0; i < n; i++)
49         cout << arr[i] << " ";
50
51     return 0;
52 }
```

```
● PS C:\Users\Induleka\Desktop\DSA\collage_training> g++ search_algo_2.cpp -o search_algo_2
● PS C:\Users\Induleka\Desktop\DSA\collage_training> .\search_algo_2.exe
Enter the number of elements: 6
Enter element 1: 120
Enter element 2: 45
Enter element 3: 78
Enter element 4: 200
Enter element 5: 60
Enter element 6: 95
Sorted array: 45 60 78 95 120 200
❖ PS C:\Users\Induleka\Desktop\DSA\collage_training> |
```

Algorithm

1. Read the number of elements **N**.
2. Read the trade values into an array.
3. Apply **Quick Sort** to sort the array in **descending order**.
4. Print the sorted array.
5. Print the first 5 elements (Top 5 values).
6. Calculate the average of the Top 5 values.
7. Calculate the overall average of all trade values.
8. Count how many values are greater than the overall average.
9. Display the required results.

```
search_algo_2.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  // Partition function for descending order
5  int partition(long long arr[], int low, int high)
6  {
7      long long pivot = arr[high];
8      int i = low - 1;
9
10     for(int j = low; j < high; j++)
11     {
12         if(arr[j] > pivot) // Descending order
13         {
14             i++;
15             swap(arr[i], arr[j]);
16         }
17     }
18
19     swap(arr[i + 1], arr[high]);
20     return i + 1;
21 }
22
23 // Quick Sort function
24 void quickSort(long long arr[], int low, int high)
25 {
26     if(low < high)
27     {
28         int pi = partition(arr, low, high);
29
30         quickSort(arr, low, pi - 1);
31         quickSort(arr, pi + 1, high);
32     }
33 }
34
35 int main()
36 {
37     int n;
38     cin >> n;
39
40     long long arr[n];
41     long long totalSum = 0;
42
43     // Input
44     for(int i = 0; i < n; i++)
```



```

search_algo_2.cpp > main()
43 // Input
44 for(int i = 0; i < n; i++)
45 {
46     cin >> arr[i];
47     totalSum += arr[i];
48 }
49
50 // Sort in descending order
51 quickSort(arr, 0, n - 1);
52
53 // Display sorted array
54 for(int i = 0; i < n; i++)
55     cout << arr[i] << " ";
56 cout << endl;
57
58 // Display Top 5 values
59 cout << "Top 5: ";
60 long long top5Sum = 0;
61
62 for(int i = 0; i < 5; i++)
63 {
64     cout << arr[i] << " ";
65     top5Sum += arr[i];
66 }
67 cout << endl;
68
69 // Average of Top 5
70 cout << "Average of Top 5: " << top5Sum / 5 << endl;
71
72 // Overall average
73 double overallAverage = (double)totalSum / n;
74
75 // Count values greater than overall average
76 int count = 0;
77 for(int i = 0; i < n; i++)
78 {
79     if(arr[i] > overallAverage)
80         count++;
81 }
82
83 cout << "Values Above Overall Average: " << count;
84
85 return 0;
86 }

```

```

PS C:\Users\Induleka\Desktop\DSA\collage_training> .\search_algo_2.exe
Enter element 2: 1200
Enter element 3: 700
Enter element 4: 2000
Enter element 5: 900
Enter element 6: 1500
Enter element 7: 3000
Enter element 8: 2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS C:\Users\Induleka\Desktop\DSA\collage_training>

```

Algorithm: Heap Sort (Ascending Order)

1. Read the number of elements **N**.
2. Read all player scores into an array.
3. Build a **Max Heap** from the array.
4. Repeat until the heap size becomes 1:
 - Swap the root (largest element) with the last element.
 - Reduce the heap size by 1.
 - Heapify the root to maintain the Max Heap property.
5. Print the sorted array in ascending order.

```
search_algo_2.cpp > heapSort(int [], int)
1  #include <iostream>
2  using namespace std;
3  // Heapify function
4  void heapify(int arr[], int n, int i)
5  {
6      int largest = i;
7      int left = 2 * i + 1;
8      int right = 2 * i + 2;
9
10     if (left < n && arr[left] > arr[largest])
11         largest = left;
12
13     if (right < n && arr[right] > arr[largest])
14         largest = right;
15
16     if (largest != i)
17     {
18         swap(arr[i], arr[largest]);
19         heapify(arr, n, largest);
20     }
21 }
22 // Heap Sort function
23 void heapSort(int arr[], int n)
24 {
25     // Build Max Heap
26     for (int i = n / 2 - 1; i >= 0; i--)
27         heapify(arr, n, i);
28
29     // Extract elements one by one
30     for (int i = n - 1; i > 0; i--)
31     {
32         swap(arr[0], arr[i]);
33         heapify(arr, i, 0);
34     }
35 }
36 int main()
37 {
38     int n;
39     cin >> n;
40
41     int arr[n];
42
43     for (int i = 0; i < n; i++)
44         cin >> arr[i];
45
46     heapSort(arr, n);
47
48     for (int i = 0; i < n; i++)
49         cout << arr[i] << " ";
50
51     return 0;
52 }
```

```
PS C:\Users\Induleka\Desktop\DSA\collage_training> g++ search_algo_2.cpp -o search_algo_2.exe
PS C:\Users\Induleka\Desktop\DSA\collage_training> .\search_algo_2.exe
Enter the number of elements: 6
Enter element 1: 850
Enter element 2: 1200
Enter element 3: 950
Enter element 4: 600
Enter element 5: 1500
Enter element 6: 1000
600 850 950 1000 1200 1500
PS C:\Users\Induleka\Desktop\DSA\collage_training>
```

Algorithm

1. Read the number of response times **N**.
2. Read all response times into an array.
3. Build a **Max Heap**.
4. Perform **Heap Sort** to sort the array in ascending order.
5. Print the sorted array.
6. The first element is the **fastest** response time.
7. The last element is the **slowest** response time.
8. Calculate the average response time.
9. Count the number of response times less than the average.
10. Calculate the percentage of cases faster than the average.
11. Display all results.

```
search_algo_2.cpp > main()
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  // Heapify function
6  void heapify(int arr[], int n, int i)
7  {
8      int largest = i;
9      int left = 2 * i + 1;
10     int right = 2 * i + 2;
11
12     if (left < n && arr[left] > arr[largest])
13     {
14         largest = left;
15     }
16
17     if (right < n && arr[right] > arr[largest])
18     {
19         largest = right;
20     }
21
22     if (largest != i)
23     {
24         swap(arr[i], arr[largest]);
25         heapify(arr, n, largest);
26     }
27 }
28
29 // Heap Sort function
30 void heapSort(int arr[], int n)
31 {
32     // Build Max Heap
33     for (int i = n / 2 - 1; i >= 0; i--)
34     {
35         heapify(arr, n, i);
36     }
37
38     // Heap Sort
39     for (int i = n - 1; i > 0; i--)
40     {
41         swap(arr[0], arr[i]);
42         heapify(arr, i, 0);
43     }
44 }
```

```
search_algo_2.cpp > main()
46  int main()
47  {
48      cout << "Enter the number of response times: ";
49      int n;
50      cin >> n;
51
52      int arr[n];
53      double sum = 0;
54
55      // Input
56      for (int i = 0; i < n; i++)
57      {
58          cout << "Enter response time " << i + 1 << ": ";
59          cin >> arr[i];
60          sum += arr[i];
61      }
62
63      // Sort the array
64      heapSort(arr, n);
65
66      // Display sorted array
67      cout << "Sorted Response Times: ";
68      for (int i = 0; i < n; i++)
69      {
70          cout << arr[i] << " ";
71      }
72      cout << endl;
73
74      // Fastest and Slowest
75      cout << "Fastest: " << arr[0] << endl;
76      cout << "Slowest: " << arr[n - 1] << endl;
77
78      // Average
79      double average = sum / n;
80      cout << fixed << setprecision(2);
81      cout << "Average: " << average << endl;
82
```

```

83     // Count cases faster than average
84     int count = 0;
85     for (int i = 0; i < n; i++)
86     {
87         if (arr[i] < average)
88         {
89             count++;
90         }
91     }
92
93     cout << "Cases Faster Than Average: " << count << endl;
94
95     // Percentage
96     double percentage = (count * 100.0) / n;
97     cout << "Percentage: " << percentage << "%" << endl;
98
99     return 0;
100 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Induleka\Desktop\DSA\collage_training> .\search_algo_2.exe
Enter response time 2: 7
Enter response time 3: 15
Enter response time 4: 9
Enter response time 5: 20
Enter response time 6: 5
Enter response time 7: 10
Enter response time 8: 8
Sorted Response Times: 5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\Users\Induleka\Desktop\DSA\collage_training>

```